Ministry of Higher Education
and Scientific Research

ACADEMIC YEAR
2025/2026

EPI DIGITAL SCHOOL
Private International Higher
School of Polytechnic

Professional Internship Report

Field of study: **Virtual Reality & Game Engineering**

Entitled

Penguin Parade: Development of a Body-Tracking VR Therapeutic Game for Children

Internship place	Author	Supervisor
Inherited Games Studio	Bouhareb Anouar	Touati Selsebil

Acknowledgements

I would like to express my sincere gratitude to Inherited Games Studio for welcoming me during this two-month internship. This experience allowed me to work in a stimulating professional environment and to develop essential skills in virtual reality development.

My heartfelt thanks go to my internship supervisor, whose guidance, availability, and constructive feedback greatly contributed to the successful completion of Penguin Parade.

Finally, I extend my appreciation to my family and close friends for their constant encouragement, patience, and support, which helped me carry out this work under the best conditions.

Table of contents

List of Figures

List of Tables

List of Abbreviations

VR: Virtual Reality

AR: Augmented Reality

SDK: Software Development Kit

GDD: Game Design Document

UI: User Interface

UX: User Experience

FPS: Frames Per Second

NPC: Non-Player Character

FBX: Filmbox (3D asset export format)

AI: Artificial Intelligence

LOD: Level of Detail

General Introduction

Virtual Reality (VR) represents one of the most dynamic and promising frontiers in interactive entertainment today, providing users with an unparalleled level of immersion and engagement. The increasing accessibility of standalone headsets such as the Meta Quest has opened the door to fully immersive experiences that combine physical interaction, intuitive controls, and creative gameplay.

This internship project, titled “Penguin Parade”, was developed within this context. The objective was to design and implement a complete VR game that uses body tracking through the Meta Movement SDK to create an experience that is both entertaining and physically engaging.

In this adventure, the player takes the role of a rescuer whose mission is to help a penguin trapped in a series of icy challenges. The game unfolds through three progressive levels dodging, leaning, and balancing each designed to test reflexes, coordination, and focus. The entire experience is optimized for Meta Quest, ensuring fluid performance, intuitive control, and player comfort.

This report presents the full development process of *Penguin Parade*, from the conceptual study and design to the technical implementation and testing stages. It aims to highlight the methodology, the technologies used, and the creative and technical challenges encountered during the project.

1. Chapter 1: Project Context and Preliminary Study

1.1 Introduction

This chapter introduces the background and objectives of *Penguin Parade*, defining its purpose, target audience, and technical scope within the broader context of VR development.

1.2 Presentation of the host organization

1.2.1 Organization Characteristics

Figure 1.1: Logo of the host company [1]

Inherited Games Studio creates educational and entertaining games, focusing on immersion through virtual reality (VR) and augmented reality (AR). Artificial intelligence personalizes the experiences by adding interactive and engaging elements.

The mission is to develop fun and educational games that combine entertainment with immersive technology, offering users enriching and interactive experiences.

The methodology is based on the use of VR/AR for immersive experiences, personalization of pathways via AI, multiplayer mode and gamification for social dynamics, and the analysis and optimization of games for a better experience. The studio focuses on designing games that are both educational and entertaining.

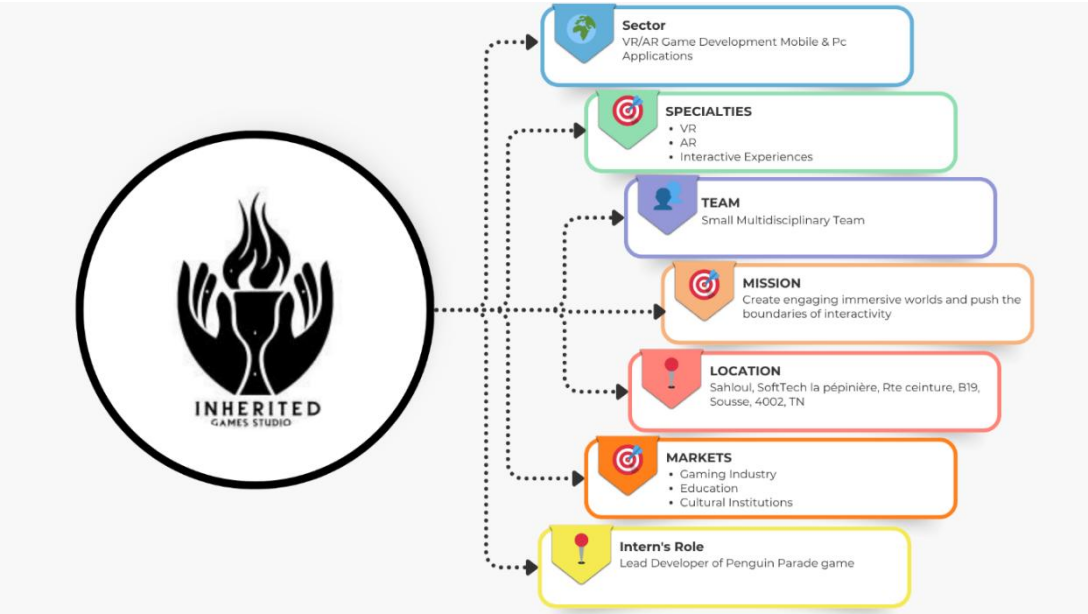

Figure 1.2: General characteristics of the host company Inherited Games Studio

1.2.2 Description of tasks performed

During my internship at Inherited Games Studio, I carried out the following main tasks:

- Implemented the core gameplay mechanics of Penguin Parade using Unity 2022.3 LTS.
- Integrated Meta Movement SDK to capture player body movements for leaning, dodging, and balancing.
- Designed and built the three game levels: Minecart Ride, Snowball Dodge, and Rope Bridge Crossing.
- Optimized 3D assets and lighting to maintain stable 72 FPS performance on the Meta Quest headset.
- Tested and calibrated the body-tracking sensitivity for better comfort and precision.
- Created and organized project tasks using Trello and ClickUp under an agile workflow.
- Used Git for version control and collaborative development tracking.

1.3 Project Presentation

Penguin Parade is a VR action–adventure game designed with Unity 2022.3 LTS, combining dynamic movement with immersive environmental interaction. The player's mission is to guide and ultimately rescue a penguin through three distinct environments:

- **Level 1 – Minecart Ride:**
  The player is seated in a moving minecart and must avoid obstacles by leaning left or right, using real body movement tracked via the Meta Movement SDK.
- **Level 2 – Snowball Dodge:**
  The player remains stationary while snowballs are launched toward them. They must quickly dodge by leaning or ducking to avoid being hit.
- **Level 3 – Rope Bridge Crossing:**
  The player must cross a narrow rope bridge by maintaining their balance using small, controlled body movements to stay upright and reach the other side.

The game was designed to showcase technical mastery of VR development, body tracking integration, and immersive gameplay design within an optimized and comfortable experience for the Meta Quest headset.

1.3.1 Objectives and Challenges

Main Objective

To demonstrate complete proficiency in VR game design and optimization by creating a fully functional experience that leverages body tracking as the primary input method.

Secondary Objectives

- Design a logical and rewarding progression system across the three levels.
- Create an immersive and engaging experience through natural player movement.
- Implement real-time body tracking using the Meta Movement SDK.
- Maintain high and stable performance (minimum 72 FPS).
- Ensure ergonomic design and comfort to avoid motion sickness.

Key Challenges

- Achieving fluid and precise movement detection using body tracking data.
- Maintaining comfort and immersion without inducing motion sickness.
- Balancing graphical quality with hardware limitations on the Meta Quest.
- Designing three unique yet mechanically consistent gameplay experiences.

1.3.2 Study of existing solutions

Before developing Penguin Parade, several well-known VR titles were analyzed to understand the best practices in immersion, motion control, and user comfort. The goal was to identify the elements that make a VR experience both engaging and comfortable, while also recognizing their limitations to design a more balanced and educational solution.

1.3.2.1 Beat Saber

Beat Saber is one of the most popular VR rhythm games. Players slice colored blocks to the beat of music using motion-tracked controllers, encouraging full-body movement and coordination.

Figure 1.3: Beat Saber [2]

Strong Points:

- Highly engaging and energetic gameplay that promotes physical activity.
- Excellent synchronization between music and action, enhancing immersion.
- Simple controls and visual feedback suitable for all audiences.

Weak Points:

- Limited variety in gameplay mechanics; repetitive over time.
- Focused only on arm movements lacks full-body or balance-based interaction.
- Not optimized for educational or training purposes, purely entertainment-driven.

1.3.2.2 Richie's Plank Experience

This experience simulates walking on a narrow plank suspended high above the ground. It tests users' sense of balance and fear of heights, leveraging psychological immersion and spatial awareness.

Figure 1.4: Richie's Plank Experience [3]

Strong Points:

- Extremely immersive and effective at triggering emotional responses.
- Simple design that effectively demonstrates VR's sense of presence.
- Good example of balance and body coordination mechanics.

Weak Points:

- Very short experience with minimal interactivity.
- Not replayable; once the player adapts to the height, immersion fades.
- No progression system or reward-based motivation.

1.3.2.3 The Climb

The Climb 2 is a climbing simulation game where players scale realistic mountains and structures using hand movements. It emphasizes upper-body coordination and physical precision.

Figure 1.5: The Climb [4]

Strong Points:

- Excellent use of environmental design and hand interaction.
- Physically engaging and visually impressive environments.
- Encourages endurance and careful coordination.

Weak Points:

- Limited to hand-based mechanics, body leaning and balance are not fully utilized.
- High hardware demand; can cause discomfort on lower-end VR systems.
- Gameplay repetition after several levels.

1.3.3 Proposed Solutions

Unlike the existing solutions, Our Proposed Solution *Penguin Parade* aims to provide a balanced combination of physical engagement, comfort, and educational value. It introduces a multi-level structure integrating different body-based mechanics dodging, leaning, and balancing within one coherent narrative experience

Key Strengths of Penguin Parade:

- Full-body interaction: integrates leaning, posture control, and balance tracking via the Meta Movement SDK.
- Varied gameplay: three distinct challenges that stimulate different physical responses (reflex, coordination, equilibrium).

- ❖ Comfort-focused: optimized movement and frame rate (72+ FPS) to prevent motion sickness.
- ❖ Educational approach: encourages physical awareness and body coordination, not just entertainment.
- ❖ Replayability: level-based structure and scoring system maintain long-term engagement.
- ❖ Performance optimization: runs smoothly on standalone Meta Quest without a PC connection.

1.3.4 Summary table

Table 1.1: Comparative table between existing solutions and the proposed solution

Feature	Beat Saber	Richie's Plank Experience	The Climb 2	Penguin Parade
Full-body interaction	✗	✗	✗	✓
Hand motion tracking	✓	✗	✓	✓
Balance control	✗	✓	✗	✓
Multi-level gameplay	✓	✗	✓	✓
Educational value	✗	✗	⚠ Partial	✓
Immersion realism	✓	✓	✓	✓
Replayability	✓	✗	⚠ Partial	✓
Motion comfort	✓	⚠ Partial	⚠ Partial	✓
Performance on standalone VR	✓	✓	✗	✓
Designed for Meta Quest	✓	✓	⚠ Partial	✓

1.4 Specification of Requirements

Before starting the implementation of Penguin Parade, it was essential to clearly define the project's requirements. This phase served to transform the initial idea into precise technical and functional objectives that would guide development. The specification of requirements describes what the system must do (functional requirements) and how well it must perform these tasks (non-functional requirements). These requirements were identified after analyzing similar VR experiences, evaluating hardware constraints of the Meta Quest headset, and discussing the expectations of the host organization, Inherited Games Studio.

1.4.1 Identification of Actors

The player Represents the main user of the game typically a child with mild balance or coordination difficulties who uses the VR experience for training and improvement.

1.4.2 Functional Requirements

Functional requirements describe the expected behaviors and features of the game — the elements that directly define the player's experience.

- Automatic player movement in the Minecart Ride level.
- Real-time leaning and body-tracking detection for dodging and balance, using the Meta Movement SDK.
- Scoring and life-management systems to monitor player performance.
- A victory sequence triggered upon rescuing the penguin.
- Smooth transitions between different levels and scenes.
- User interface (UI) elements for score, timer, and level information, optimized for readability in VR.

1.4.3 Non-Functional Requirements

Non-functional requirements concern the quality of the system — its performance, comfort, usability, and overall user experience.

- **Performance:** Maintain a stable frame rate of 72 FPS or higher on the Meta Quest headset.

- **Comfort:** Prevent motion sickness through linear, controlled movement and soft camera transitions.
- **Immersion**: Provide responsive visual, auditory, and haptic feedback to reinforce player actions.
- **Maintainability:** Code organized in modular scripts for easier debugging and future extensions.
- **Accessibility**: Simple menu navigation and readable in-game text for diverse audiences.

1.5 Work Methodology

The development of Penguin Parade followed an Agile approach, emphasizing flexibility, iteration, and continuous feedback. This method suited the short two-month internship and the experimental nature of VR development, where frequent adjustments were needed to optimize comfort and performance. Within the Agile philosophy, the Scrum framework was used to structure the workflow through short, weekly sprints. Scrum defines three key roles:

- Product Owner: represents the client or project supervisor, defining objectives and priorities.
- Scrum Master: ensures the proper application of the framework and facilitates communication.
- Development Team: carries out implementation tasks and reports progress at the end of each sprint.

In the context of this internship, the project was divided into weekly sprints, each lasting one week, covering a total of eight sprints over two months. At the beginning of each sprint, objectives were set jointly with the internship supervisor. At the end of the week, a meeting was held to present the progress achieved, discuss encountered difficulties, and adjust the next sprint's priorities.

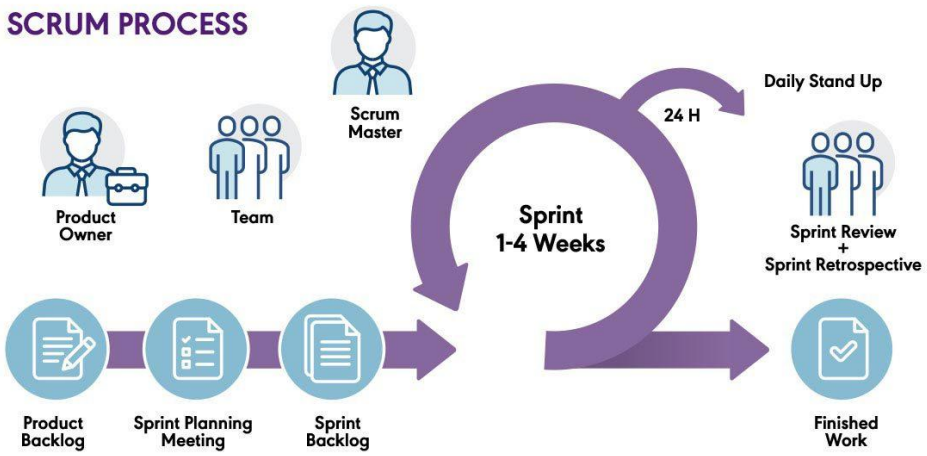

Figure 1.6: Illustration of the Scrum framework [5]

The project's backlog, sprint tasks, and progress were managed through ClickUp, which provided both sprint and timeline views to track development across the eight weeks of the internship. Each sprint covered a specific stage of the project, from environment setup and SDK integration to gameplay implementation, testing, and optimization. Although no formal Gantt chart was created, ClickUp's visual tools offered a clear overview of objectives, milestones, and progress, maintaining flexibility while ensuring structured, transparent project management consistent with Agile and Scrum principles.

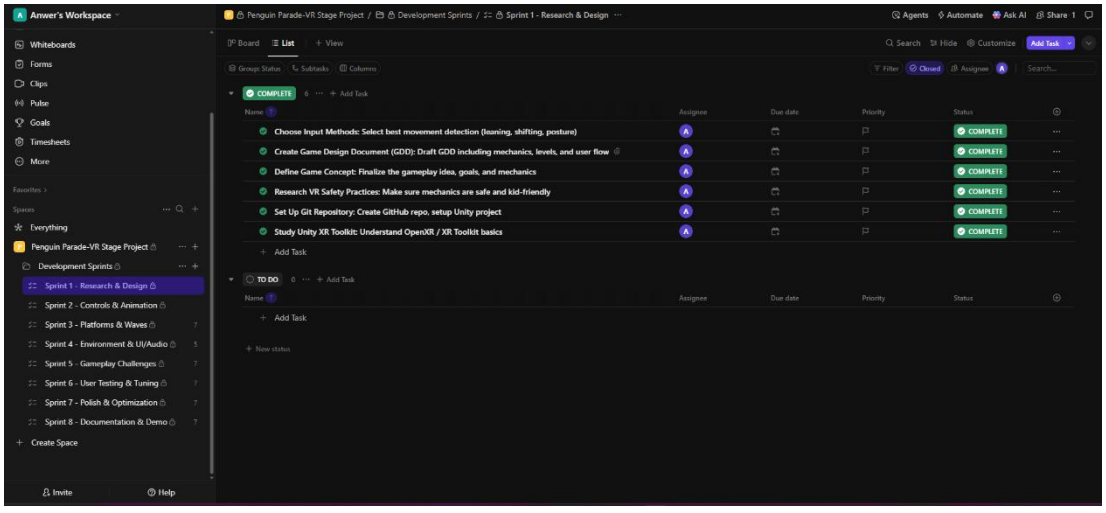

Figure 1.7: List of project development stages [6]

1.6 Conclusion

This chapter established the foundational context and methodology for the Penguin Parade project. By analyzing existing VR solutions and identifying key requirements, an immersive and educational experience was defined, leveraging VR technology. The Agile approach, combined with Scrum, provided a flexible framework for iterative development, ensuring regular adjustments based on progress and feedback. The project is now ready to move into the detailed design and implementation phase.

2. Chapter 2: Conceptual Study

2.1 Introduction

This chapter presents the conceptual study and design foundations of the Penguin Parade project.

It begins with the elaboration of the Game Design Document (GDD), which defines the objectives of the experience, the target user profile, the interactive mechanics, and the pedagogical intent behind the gameplay. The chapter then describes the architectural and system design, including the main components of the application and their relationships.

Next, the user interactions and gameplay flow are modeled through UML diagrams (use case and activity diagrams), illustrating the player's progression and the system's behavior in different situations. The chapter also covers the design of virtual environments, focusing on the composition of levels, spatial organization, and visual identity that support immersion and comfort.

Finally, the technological stack, hardware requirements, and anticipated technical challenges are presented, followed by a summary of key design decisions that guided the implementation phase.

2.2 Game Design Document (GDD) Overview

Before beginning the technical phase, the creation of a Game Design Document (GDD) was an essential step in defining the conceptual framework of Penguin Parade.

This document served as a blueprint for the project, ensuring coherence between the pedagogical objectives, the entertainment aspects, and the technical constraints of the Meta Quest headset. The GDD provided a structured representation of the user experience, the interaction mechanics, and the criteria for performance and comfort that guided every stage of development.

2.2.1 Objectives of the Experience

The primary goal of Penguin Parade is to design a virtual reality experience that is both educational and entertaining, fostering the development of motor coordination, reflexes, and balance among children.The game was conceived not merely as a source of entertainment, but as a therapeutic and cognitive training tool that stimulates physical engagement through natural body movements.

The project also aimed to demonstrate the potential of VR and body tracking in the context of health, rehabilitation, and education.Through a series of progressively challenging activities, the player develops essential motor skills while being immersed in a motivating and rewarding environment.The use of body tracking reinforces the connection between real-world movement and virtual interaction, creating a form of "active learning through play."

2.2.2 Target Audience

The experience was designed primarily for children between 7 and 12 years old, particularly those facing mild motor or balance disorders.The design process considered both physiological and cognitive factors of this age group: reduced attention span, variable coordination skills, and sensitivity to visual motion.To ensure comfort and accessibility, the interactions were simplified and limited to essential movements such as leaning and balancing, which are natural and low-risk gestures for children.

The game's duration and rhythm were also adjusted to maintain engagement while avoiding fatigue.Sessions are short and progressive, with immediate positive feedback visual, auditory, and sometimes haptic encouraging the player to repeat actions and improve performance.This user-centered approach ensures that Penguin Parade remains inclusive and suitable for children under the supervision of therapists, educators, or family members.

2.2.3 Core Gameplay Pillars

The Penguin Parade GDD is structured around three conceptual pillars that define the gameplay experience and its learning impact:

- **Physical Interaction:** All player actions are controlled through natural body movements tracked by the Meta Movement SDK, including leaning, side-

stepping, and postural adjustments. This design promotes kinaesthetic learning, where the body becomes the main interface of interaction.

- **Progressive Challenge:** The game structure is organized into three sequential levels, each emphasizing a distinct motor skill (reaction, coordination, and balance). The difficulty curve is calibrated to maintain motivation while progressively testing the player's stability and control.
- **Comfort and Feedback:** Every element from scene lighting to camera movement was designed to ensure VR comfort. Audio cues and world-space UI elements provide instant and intuitive feedback, reinforcing the player's sense of presence and accomplishment.

2.2.4 Player Journey

The player embodies a rescuer whose mission is to help a trapped penguin by completing three immersive physical challenges. Each stage of the journey corresponds to a level that emphasizes a specific motor objective:

- **Level 1 – Minecart Ride:** The player travels automatically along a rail track and must lean left or right to dodge obstacles. This level focuses on reflex development and introduces the basic body-tracking controls.
- **Level 2 – Snowball Dodge:** The player remains stationary while snowballs are launched toward them. They must anticipate direction and timing to dodge successfully. This stage enhances reaction speed and spatial awareness.
- **Level 3 – Rope Bridge Crossing:** The player must cross a suspended bridge without falling, maintaining stability through subtle posture adjustments. This final level targets balance control and fine motor coordination.

Each level ends with an animation sequence and visual reward, creating a sense of achievement and motivating the player to progress further.

2.2.5 Pedagogical and Entertainment Goals

The educational approach behind Penguin Parade relies on learning through movement. By immersing the player in a game that requires real physical actions, the experience stimulates both motor and cognitive processes. The player improves coordination and balance while also developing attention, anticipation, and self-confidence.

From an entertainment perspective, the colorful and dynamic design encourages playful participation. The inclusion of sound effects, environmental feedback, and light narrative elements (the penguin rescue theme) keeps the experience engaging yet meaningful. In this way, Penguin Parade bridges the gap between therapy and play, aligning with the broader philosophy of "serious games."

2.2.6 Use of the GDD During Development

The Game Design Document was not a static document but a living reference throughout the project. It was continuously refined after each sprint review to reflect changes, player feedback, and technical discoveries. Each update helped ensure consistency between the intended experience and the implemented features.

The GDD was also used to coordinate communication with the internship supervisor, to define sprint objectives, and to evaluate progress. This iterative refinement process allowed the game to evolve naturally while staying faithful to its educational and technical foundations.

2.3 General Architecture of the Game

The architecture of Penguin Parade was designed around three main priorities: clarity, modularity, and performance optimization all essential for virtual reality development on a standalone headset such as the Meta Quest.

The project follows a scene-based modular structure in Unity 2022.3 LTS, allowing each level (Minecart Ride, Snowball Dodge, and Rope Bridge Crossing) to function as an independent environment.

This separation simplifies development and testing while enabling future extensions, such as new challenges or rehabilitation modes.

A single controller, the GameManager, coordinates global gameplay elements like score, life count, and progression state.

It also handles level transitions and communicates with the rest of the system through a lightweight event-based architecture, implemented as a Singleton to ensure persistence across scenes.

2.3.1 Core Components

The main subsystems of the architecture are summarized below:

- **GameManager:** Supervises global flow, scoring, and level transitions.
- **PlayerBalanceController / PlayerBalanceController2:** Converts posture data from the Meta Movement SDK into in-game actions, defining sensitivity thresholds for comfort and precision.
- **Level Managers:** Control level-specific logic such as obstacle spawning, collision checks, or balance evaluation while communicating with the GameManager.
- **UI Managers:** Display world-space information (score, timer, instructions) optimized for visibility and ergonomic placement in VR.
- **Physics and Interaction Layer:** Relies on Unity's physics system to detect motion and collisions, ensuring real-time feedback and immersion.

Together, these components form a clear, maintainable hierarchy that balances technical efficiency with player experience.

2.3.2 Communication and Data Flow

The communication between scripts follows a top-down logic controlled by the GameManager.

Each level manager reports to it via events, while secondary systems such as UI, audio, and scoring modules subscribe to these events to update their states. This event-driven architecture minimizes dependencies and improves maintainability. For example, when the player successfully dodges an obstacle, the event OnSuccessfulDodge() triggers updates in the ScoreManager, plays an audio cue, and refreshes the on-screen score simultaneously.

This structure not only simplifies debugging but also facilitates parallel development between gameplay and UI components an essential feature in a small-scale production context like an internship project.

2.3.3 Optimization for Meta Quest

Performance optimization was a central architectural concern. The Meta Quest operates independently, without PC assistance, so all rendering and calculations must remain

lightweight. To ensure smooth execution at 72 frames per second, several optimization techniques were integrated directly into the architecture:

- Use of low-poly models and compressed textures to reduce GPU load.
- Implementation of baked lighting instead of real-time shadows to limit CPU usage.
- Application of Occlusion Culling to prevent unnecessary rendering of hidden objects.
- Avoidance of costly physics calculations in Update() functions by preferring event-based triggers or FixedUpdate() calls.

These techniques ensured a stable frame rate across all levels while maintaining visual quality and user comfort.

2.3.4 Architectural Synthesis

In summary, the architecture of Penguin Parade follows a modular and event-driven design, perfectly adapted to the requirements of immersive VR applications. Its structure guarantees clarity, scalability, and optimized performance, making it possible to focus on the educational and interactive goals of the project rather than technical limitations.

This conceptual foundation prepared the ground for the next phase: the detailed level design and gameplay implementation, where each environment transforms these architectural principles into interactive and pedagogical experiences.

2.4 Level Design and Gameplay

The gameplay of Penguin Parade was structured around three short, progressive challenges that gradually develop the player's motor skills reflexes, coordination, and balance.

Each level was conceived as a self-contained learning situation, using the same movement-based control logic but introducing a distinct spatial context and objective. This progression ensures that the player experiences a feeling of accomplishment while also exercising specific physical abilities targeted by the pedagogical design.

All levels rely on the Meta Movement SDK, which captures the player's posture, leaning angle, and lateral movement to translate real actions into in-game responses. The game avoids the use of controllers, promoting a more natural, body-driven interaction suited for children and

rehabilitation contexts. Visual feedback, sound cues, and score indicators guide the player and reinforce correct motion execution.

2.4.1 Level 1 – Minecart Ride

The first level introduces the player to basic movement mechanics in a dynamic but controlled setting.

Seated in a moving minecart that travels along a predefined spline track, the player must lean left or right to avoid approaching obstacles. The PlayerObstacleInteraction component compares the real-world body tilt with the expected direction and validates a successful dodge when both coincide. This introductory challenge focuses on reaction time and lateral control. The environment's speed and the frequency of obstacles were carefully balanced to stimulate reflexes without causing discomfort.

Visual tunnel effects and rhythmic sound feedback enhance immersion while keeping the experience playful and safe

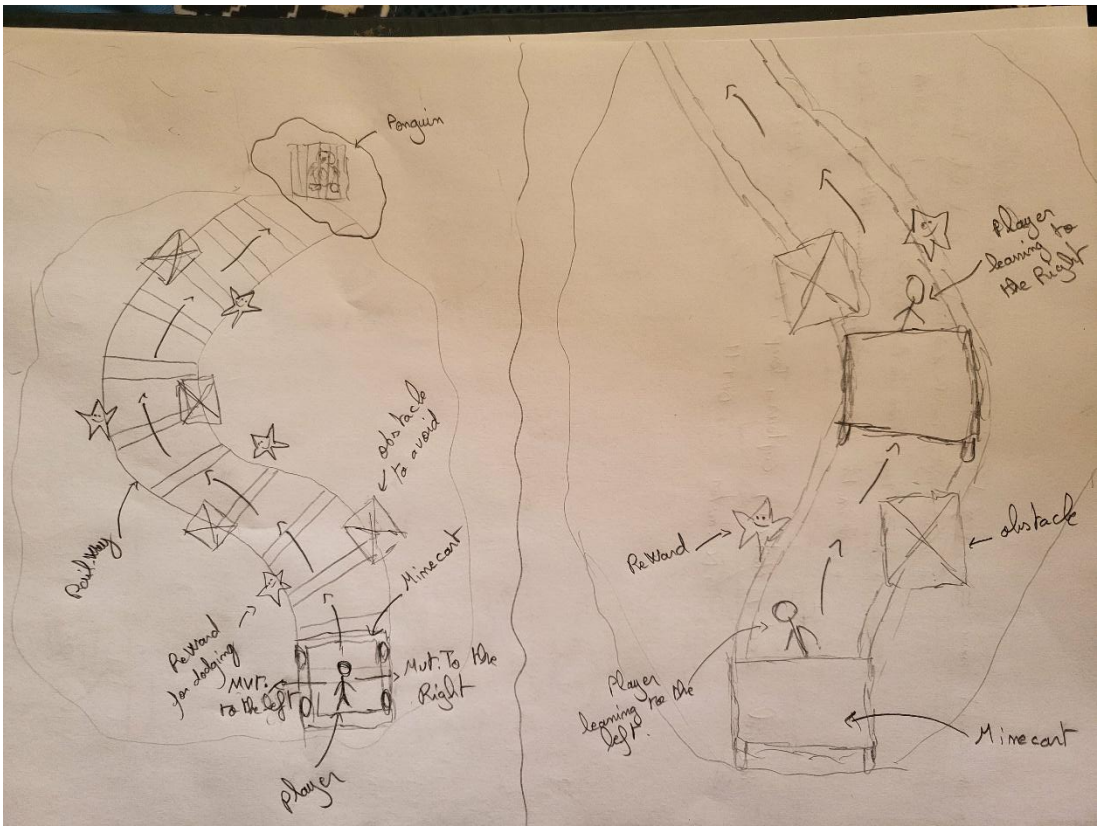

Figure 2.1: Minecart Ride Concept Sketch

2.4.2 Level 2 – Snowball Dodge

The second level intensifies the physical engagement while keeping the player stationary in space. From a fixed position, the player faces multiple SnowballCannons that launch projectiles at irregular intervals and trajectories. Using leaning or slight crouching motions, the player must dodge incoming snowballs detected through colliders managed by the DodgeDetector script.

The goal here is to train anticipation and whole-body coordination. Each successful dodge increases the score, while a hit triggers a short vibration and visual cue but never penalizes harshly, maintaining motivation, producing a demanding but still comfortable challenge for young users.

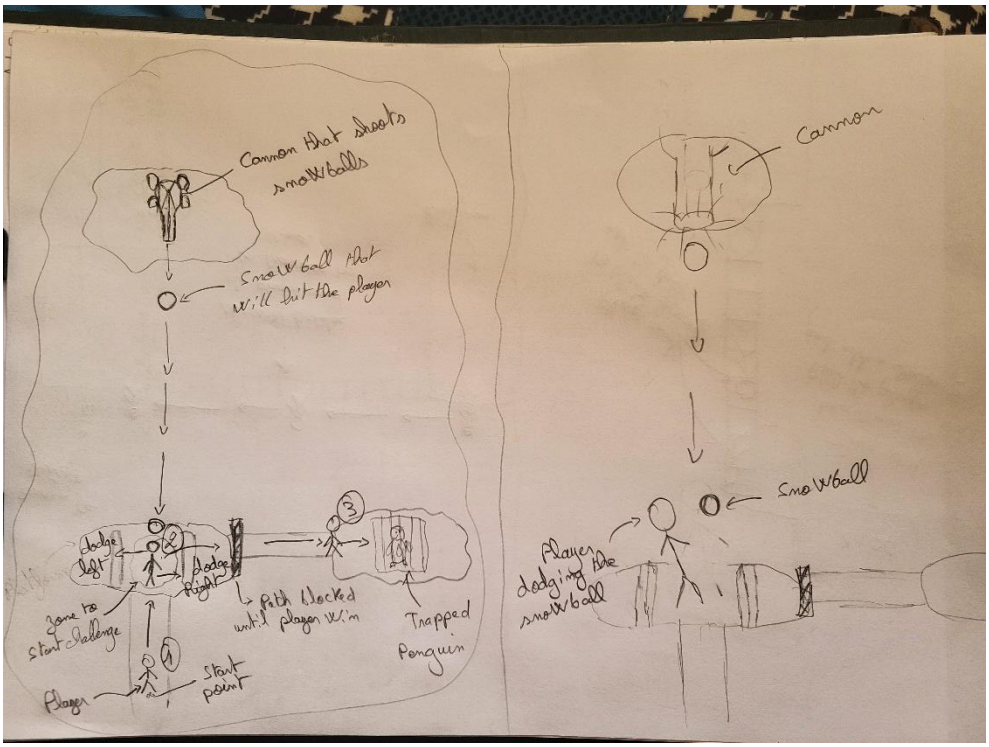

Figure 2.2: Snowball Dodge Concept Sketch

2.4.3 Level 3 – Rope Bridge Crossing

The final stage centers on postural balance and body stability. The player stands on a suspended bridge that gently sways, requiring continuous micro-adjustments to remain upright. Through the PlayerBalanceController2, the system tracks hip rotation and lateral tilt in real time; exceeding predefined thresholds results in a loss of equilibrium and a “fall” animation.

This level represents the culmination of the experience, combining the reflexes learned earlier with controlled precision. Its objective is to develop fine motor control and sustained concentration, two abilities crucial in pediatric balance training. The challenge ends successfully when the player maintains stability until the countdown timer reaches zero, rewarding perseverance with celebratory sound and animation sequences.

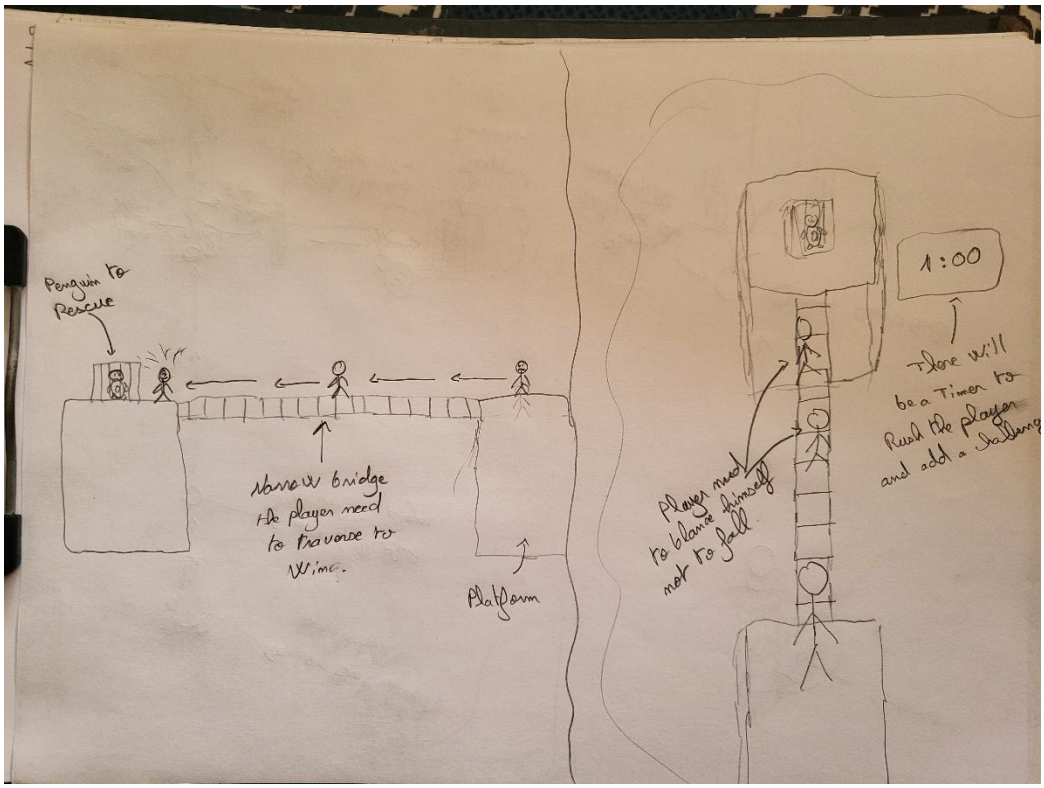

Figure 2.3: Rope Bridge Crossing Concept Sketch

2.4.4 Design Rationale

Each level was deliberately kept short between one and two minutes to maintain engagement and avoid fatigue. The environments were designed with soft colors, wide spaces, and limited visual complexity to reduce sensory overload and motion sickness risk. Gameplay progression was calibrated according to pedagogical difficulty, moving from reflexive response (Minecart) to coordination (Snowball) and finally equilibrium (Bridge).

This structure transforms the game into a mini-curriculum of body awareness, where fun and training coexist. By combining immersive design with educational purpose, Penguin Parade demonstrates how VR can be used not only as entertainment but also as a tool for motor rehabilitation and cognitive engagement.

2.5 UML Diagrams and System Modeling

To ensure a structured and coherent design process, Penguin Parade was modeled using Unified Modeling Language (UML) diagrams. These diagrams serve as abstract representations of the system's logic, interactions, and gameplay flow. They were used to maintain consistency between the pedagogical objectives of the game and its technical implementation in Unity.

Two main diagrams were created during the conceptual phase: The Use Case Diagram and the Activity Diagram. Together, they describe both what the player can do and how the system responds to each action.

2.5.1 Use Case Diagram

The use case diagram presented below provides an overall view of the main interactions between the central actor of the application the Player (child) and the various functionalities offered within the virtual environment of Penguin Parade.

It highlights the essential gameplay use cases, distinguishing between core actions (such as dodging, leaning, and maintaining balance) and supporting functionalities (such as visual and audio feedback, level progression, and automatic retries). Each use case represents an interactive feature accessible to the player through natural body movements, captured in real time by the Meta Movement SDK. The relationships marked with <<extend>> show optional or complementary actions that may occur depending on the gameplay situation for instance, leaning left or right during obstacle avoidance, or crouching in the snowball challenge.

This diagram structures the functional design of Penguin Parade and serves as a conceptual foundation for understanding how the player's physical actions influence the internal logic of the system. It also reinforces the pedagogical intent of the project, which combines entertainment and motor skill development through immersive VR mechanics.

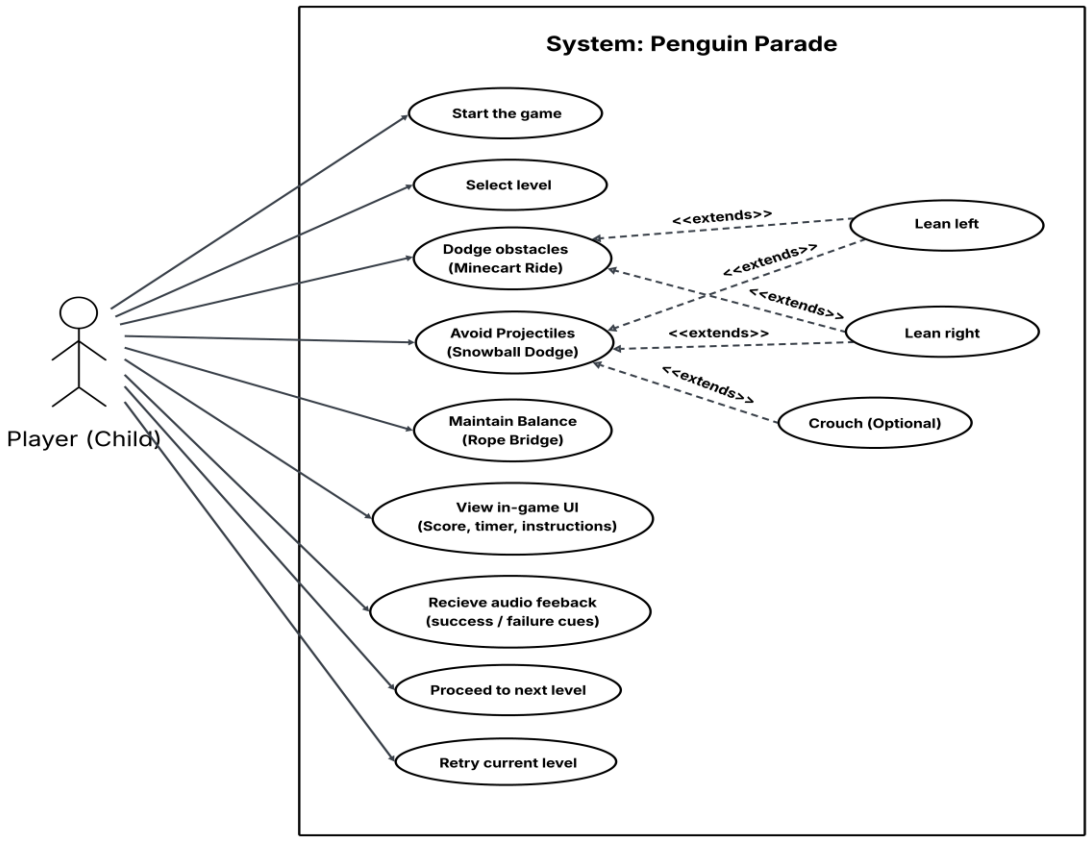

Figure 2.4: Use case Diagram of Penguin Parade

2.5.2 Activity Diagram

The activity diagram shown below models the overall flow of a play session in Penguin Parade. It depicts the chronological sequence of actions performed by the player and the system's automatic responses, from the moment the game is launched to the return to the main menu after completing all levels.

The diagram highlights the logical progression of the experience: initialization, level loading, gameplay monitoring through body tracking, and automatic transitions between success or failure states. In case of success, the system automatically loads the next level; in case of failure, the same level restarts immediately, ensuring a fluid and uninterrupted experience.

This UML representation was used to structure and validate the gameplay flow within Unity, ensuring that all game states and transitions were coherent and responsive. It also supported the implementation of asynchronous scene management and event-based scripting, which were key to achieving smooth performance on the standalone Meta Quest headset.

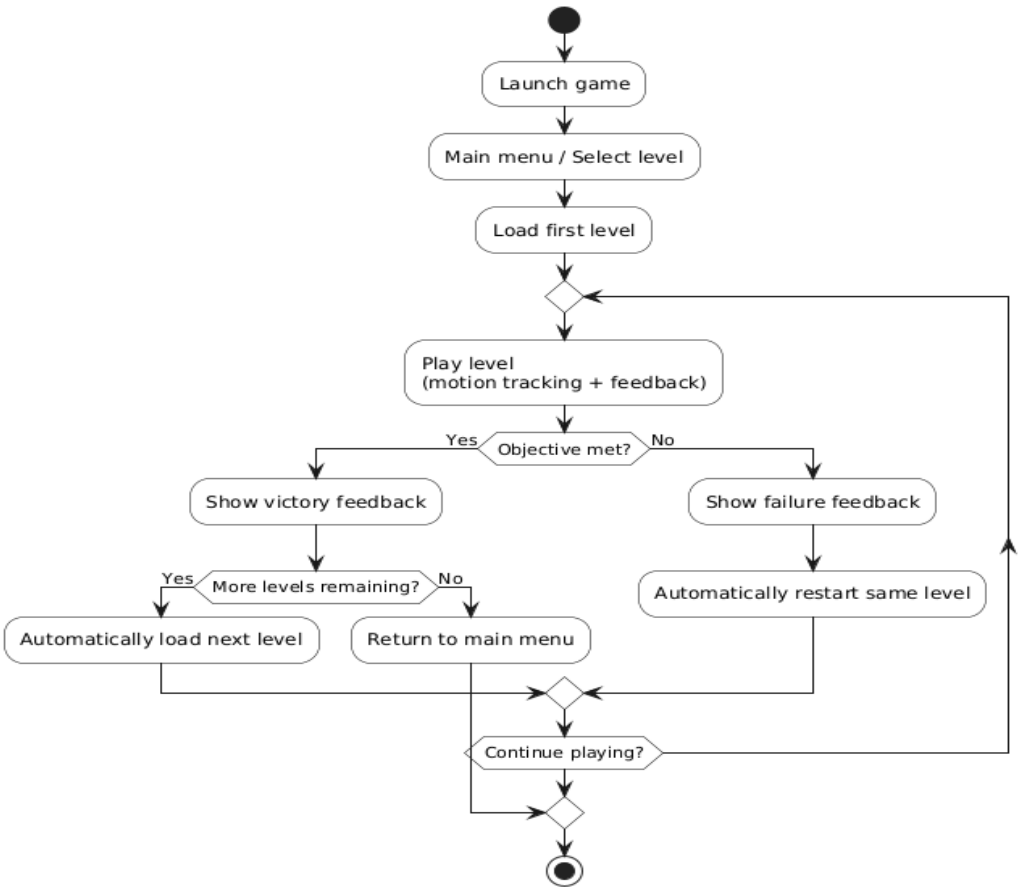

Figure 2.5: Activity Diagram of Penguin Parade

2.6 Choice of Technologies and Tools

The technologies and tools used for developing Penguin Parade were selected to ensure performance, stability, and full compatibility with the Meta Quest 2/3 standalone headset. Each software contributed to a specific phase of the project from 3D modeling and gameplay scripting to sprint management, version control, and sound design ensuring a cohesive and optimized development pipeline.

2.6.1 Hardware Used

The development of Penguin Parade was carried out on a Windows-based PC equipped with a modern processor and dedicated graphics card to ensure efficient performance during real-time testing and scene rendering. This workstation allowed the smooth execution of Unity 2022.3.56 LTS, Blender, and other production tools without latency, supporting rapid iteration and light baking processes. The high processing capability was essential for compiling builds and profiling the game before deployment to the VR headset.

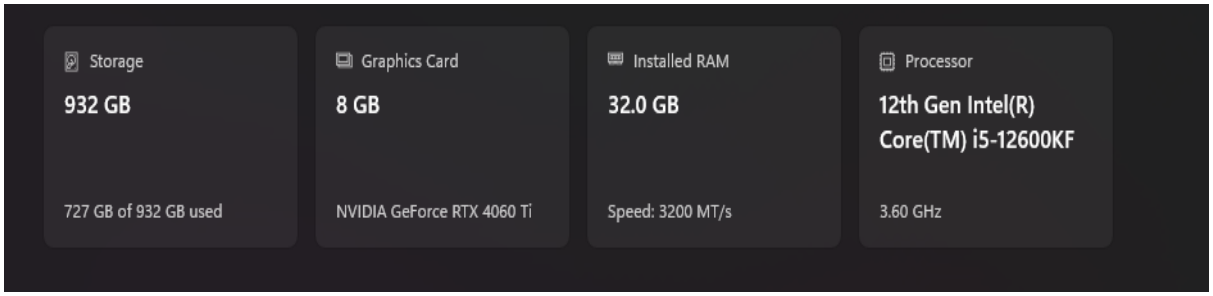

Figure 2.6: Development PC used during production

Meta Quest 3

Figure 2.7: Meta Quest 3 headset used for development and testing [7]

The Meta Quest 3 headset was chosen as the primary platform for development and testing. Its wireless design and advanced motion tracking capabilities made it ideal for an experience centered on body movement and postural balance. The Quest 3 offers strong standalone performance, allowing smooth execution of 3D scenes at 72 FPS without the need for a connected PC. The game was optimized specifically for its hardware constraints using baked lighting, compressed textures, and occlusion culling to ensure comfort and visual fluidity.

2.6.2 Technologies and Tools

Unity 2022.3.56 LTS

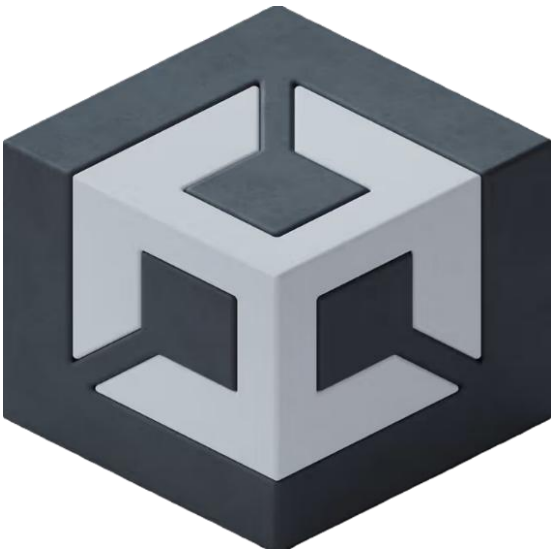

Figure 2.8: Logo of Unity [8]

The project was entirely developed using Unity 2022.3.56 LTS, chosen for its long-term stability and integrated VR support. Unity's component-based architecture and real-time editor allowed rapid prototyping and iteration of gameplay mechanics. C# scripting was used to implement core systems such as the GameManager, Level Managers, and interaction scripts. The engine's compatibility with Meta's SDKs made it the ideal environment for building a high-performance and immersive experience on the Quest 3.

Meta XR All-in-One SDK

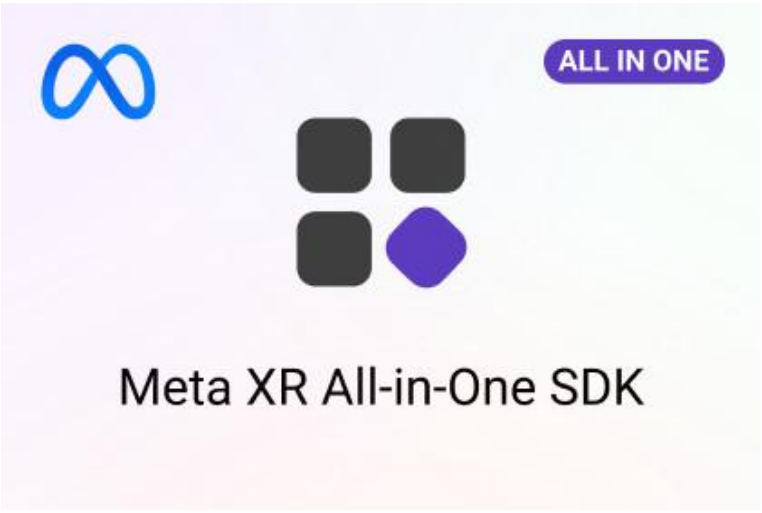

Figure 2.9: Meta XR All-in-One SDK package for Unity [9]

The Meta XR All-in-One SDK was integrated to enable full headset functionality on the Meta Quest 3. It handled essential VR features such as camera tracking, input recognition, and device optimization through OpenXR. This SDK provided seamless integration between Unity and the Quest hardware, ensuring accurate motion capture and interaction responsiveness within the VR environment.

Meta Movement SDK

Figure 2.10: Meta Movement SDK used for body tracking [10]

The Meta Movement SDK was fundamental to the project, as it enabled body tracking the core mechanic of Penguin Parade. It captured the player's real-time posture, tilt, and leaning movements and converted them into in-game actions such as dodging, crouching, or balancing. The SDK's reliability and low latency were crucial in maintaining natural motion feedback and avoiding discomfort for the player.

Blender

Figure 2.11: Blender logo [11]

Blender was used to modify and optimize 3D assets imported from Sketchfab. The software enabled mesh reduction, UV adjustments, and light baking to meet the performance requirements of standalone VR hardware. Its open-source nature and compatibility with Unity's FBX export format made it an efficient solution for quick asset refinement and environment adjustments.

Sketchfab

Figure 2.12: Skecthfab logo [12]

The majority of 3D assets were imported from Sketchfab, a vast online library of free and licensed 3D models. This approach allowed rapid content creation while maintaining artistic consistency across the three levels of the game. All downloaded assets were optimized,

rescaled, and occasionally edited in Blender to fit Unity's performance constraints and the game's low-poly aesthetic.

GitLab

Figure 2.13: Gitlab logo [13]

GitLab was used as the version control platform throughout the development. It ensured safe storage of the Unity project and allowed version tracking and rollback when needed. Using GitLab provided structure and reliability during iterative testing phases, reducing risks of data loss and ensuring proper documentation of changes.

ClickUp

Figure 2.14: Clickup logo [6]

Project organization and sprint management were handled using ClickUp. Each week represented a sprint containing defined objectives, assigned tasks, and review sessions with the

internship supervisor. ClickUp facilitated clear planning, real-time tracking, and transparent reporting of development progress, ensuring the project advanced consistently through its two-month schedule.

Lucidchart

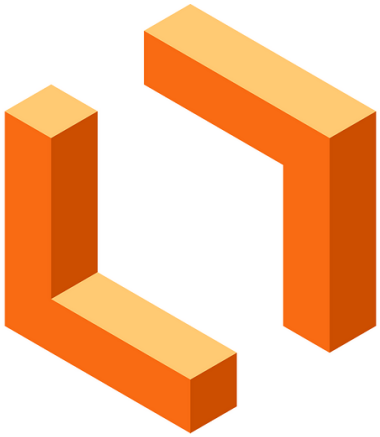

Figure 2.15: Lucidchart logo [14]

All UML diagrams and system flowcharts were created using Lucidchart. This web-based tool allowed the design of the use case and activity diagrams through a clean, intuitive interface. Lucidchart's export options (PNG, PDF, SVG) made it simple to integrate diagrams directly into the report and maintain visual clarity.

Audacity and ElevenLabs

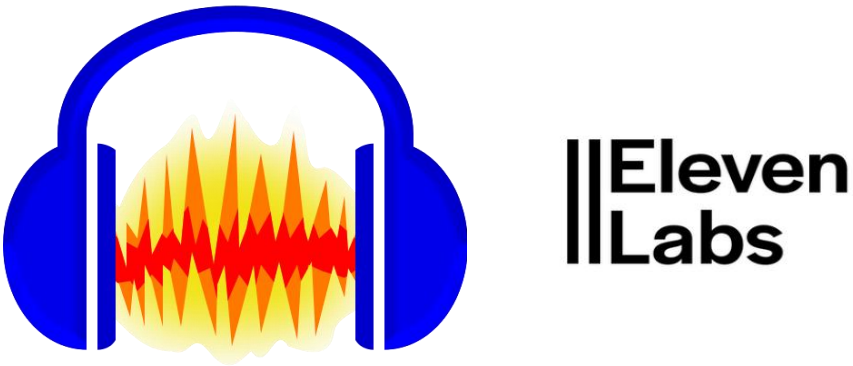

Figure 2.16: Logos of Audacity and ElevenLabs used for sound design [15] [16]

For the sound design, Audacity was used to record, edit, and process audio clips such as victory sounds, balance warnings, and ambient effects. In addition, ElevenLabs, an AI-based text-to-speech platform, was used to generate short vocal cues and sound effects, giving the game a more dynamic and polished auditory experience. Both tools contributed to the immersive feedback system that reinforces the player's physical actions within the VR environment.

2.7 Anticipated Technical Constraints and Challenges

The development of a virtual reality game designed for a standalone headset such as the Meta Quest 3 presents several specific technical constraints. From the early stages of the project, multiple challenges were identified and considered when defining the game's design and architecture.

Optimization for Standalone Hardware

Unlike PC-based VR applications, the Meta Quest 3 operates independently, with limited CPU and GPU resources.

To ensure smooth performance, several optimization strategies were adopted:

- Reduction of polygon counts in 3D models.
- Use of compressed textures to lower memory consumption.
- Preference for baked lighting instead of real-time calculations.
- Avoidance of heavy post-processing effects.
- Implementation of occlusion culling to skip rendering of hidden objects.

These techniques were crucial to achieving a stable and fluid build optimized for the headset's capabilities.

Ergonomics and User Comfort

To minimize fatigue or discomfort during gameplay:

- Player movement was designed around natural body gestures rather than artificial locomotion.
- The camera and UI were placed at ergonomic height and distance for clear visibility.

- The visual environment was simplified to avoid clutter and reduce motion sickness risk.

Natural and Intuitive Interactions

- The body-tracking system was fine-tuned to respond smoothly to leaning and balancing motions.
- Visual and auditory feedback was integrated to reinforce user engagement.
- Interaction cues were designed to be intuitive and context-aware, ensuring accessibility for younger players.

Development Autonomy

As a solo development internship, the project required managing multiple disciplines simultaneously, including 3D optimization, C# scripting, user experience, and gameplay testing. Continuous self-learning was necessary to troubleshoot SDK integration issues and achieve a polished final prototype.

2.8 Conclusion

This chapter presented the conceptual and technological framework that guided the development of Penguin Parade. It detailed the gameplay foundations, architectural structure, and the selection of software tools and SDKs used to ensure both pedagogical relevance and technical efficiency. The anticipated challenges highlighted the need for precise optimization and thoughtful design choices adapted to the limitations of standalone VR hardware.

These conceptual foundations laid the groundwork for the next stage the design and implementation phase where theoretical principles were translated into a functional and interactive VR prototype, merging entertainment and physical engagement within a fully immersive environment.

3. Chapter 3: Realization and Implementation

3.1 Introduction

This chapter describes the practical realization of Penguin Parade, from asset preparation to integration, testing, and delivery on Meta Quest 3. Built over eight weekly sprints, the implementation favored an efficient pipeline suited to a two-month internship: curated assets from Sketchfab refined in Blender, modular Unity scenes per level, and event-driven scripts connected to body tracking via the Meta Movement SDK.

The chapter first outlines the asset preparation pipeline (selection, optimization, export), then details the implementation of each level Minecart Ride, Snowball Dodge, and Rope Bridge Crossing highlighting the specific mechanics and validation logic used in each case. It then presents the core systems (GameManager, level managers, UI/audio feedback, body-tracking calibration), followed by a short inventory of assets effectively used in the prototype.

Finally, the chapter summarizes validation and testing, performance optimization for standalone VR (72 FPS target), and build/export settings for Quest, establishing a transparent view of how design intentions were translated into a functional, comfortable VR experience.

3.2 Asset Preparation Pipeline

The creation of Penguin Parade began with building a reliable and optimized asset workflow. Since the internship's timeframe was limited, efficiency and performance were prioritized over full-scale custom modeling. Therefore, a hybrid approach was adopted combining free and licensed assets from Sketchfab with minor adjustments in Blender to ensure coherence, scalability, and compatibility with VR constraints.

3.2.1 Sourcing and Editing in Blender

Assets were first selected on Sketchfab according to polycount, style consistency, and license permissions. Once imported into Blender, they underwent a lightweight optimization process:

- Cleaning unnecessary meshes or hidden faces.
- Aligning pivots and resetting transformations for correct placement in Unity.
- Adjusting scale and proportions for consistent visual coherence across all levels.

❖ Applying basic UV corrections and material merging to reduce draw calls.

This process guaranteed a uniform visual style throughout the three environments and ensured that all assets remained lightweight enough to sustain the 72 FPS target on Meta Quest 3.

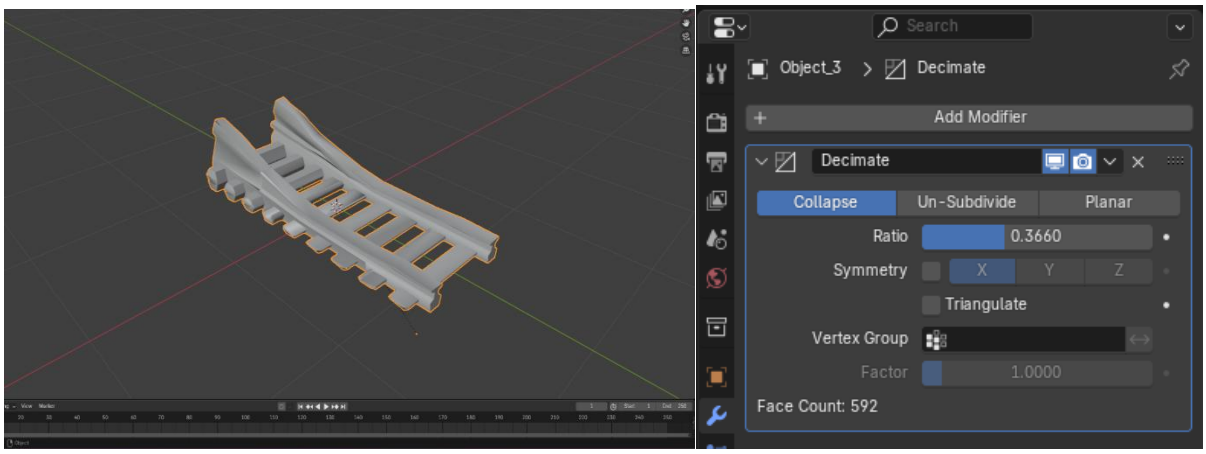

Figure 3.1: Optimization of imported Sketchfab assets in Blender

3.2.2 Export and Integration in Unity

Once optimized, models were exported as FBX files with correct scale settings and orientation, then imported into Unity. Each element was converted into a prefab with appropriate colliders and material assignments. Unity's built-in shaders were used instead of complex materials to minimize GPU load.

The prefabs were organized in structured folders (Assets/Models, Assets/Prefabs, Assets/Materials, etc.) for easy maintenance and scene integration.

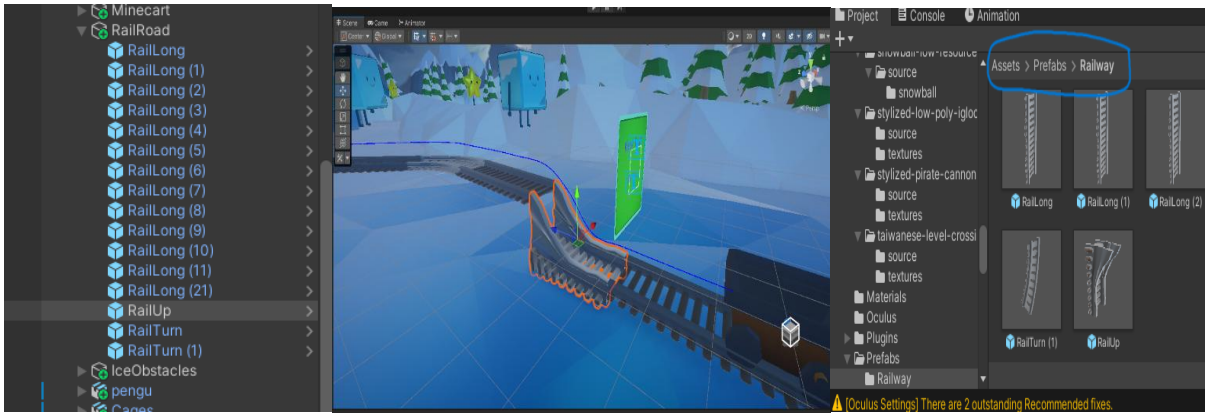

Figure 3.2: Asset prefab setup and organization in Unity

3.3 Level Implementation

Each level of Penguin Parade was designed to explore a different physical interaction principle dodging, reacting, and balancing while maintaining coherence in gameplay rhythm and visual tone. The levels were developed as independent Unity scenes to facilitate testing and debugging, then connected through a centralized GameManager.

The Meta Movement SDK was central to all stages of development. It provided real-time data from the player's body, such as hip position and tilt, which were used to drive in-game reactions. Each scene combined simple mechanics, optimized environments, and immediate feedback to promote immersion without inducing discomfort.

3.3.1 Main Menu

The experience begins in an immersive main menu environment set inside an icy-themed room, consistent with the game's overall arctic atmosphere. Instead of traditional UI buttons, the menu takes the form of four physical signboards, each representing a different level (Minecart Ride, Snowball Dodge, Rope Bridge Crossing, and a Quit game).

When the player approaches one of the signs, a circular loading indicator appears above it and begins to fill progressively. Once the circle is completely filled, the corresponding level scene loads automatically. This interaction replaces button clicks with a proximity-based trigger, making the selection process intuitive and controller-free ideal for a body-tracking experience.

The environment includes subtle animations, ambient lighting, and background audio to enhance immersion. This approach provides an organic, natural entry point to the gameplay experience, reinforcing the sense of presence right from the first moments in VR.

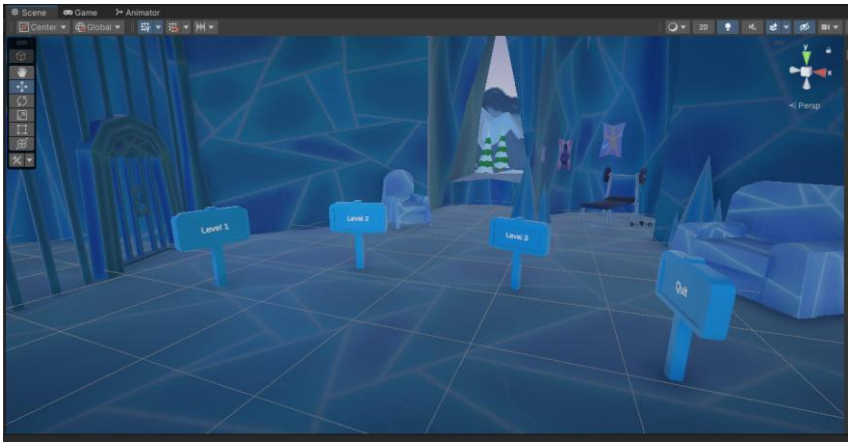

Figure 3.3: Main menu environment with interactive level-selection signs

3.3.2 Level 1 – Minecart Ride

The opening level places the player in a fast-moving minecart, traveling along a predefined track built using Unity's Splines system. Obstacles appear at varying distances, forcing the player to lean left or right to avoid collisions.

A dedicated script (SplineMinecartController) controls the minecart's movement, while the PlayerLeanDetector interprets data from the Meta Movement SDK to determine if the player has leaned correctly. Successes trigger sound and visual feedback, while failed dodges result in temporary screen color changes.

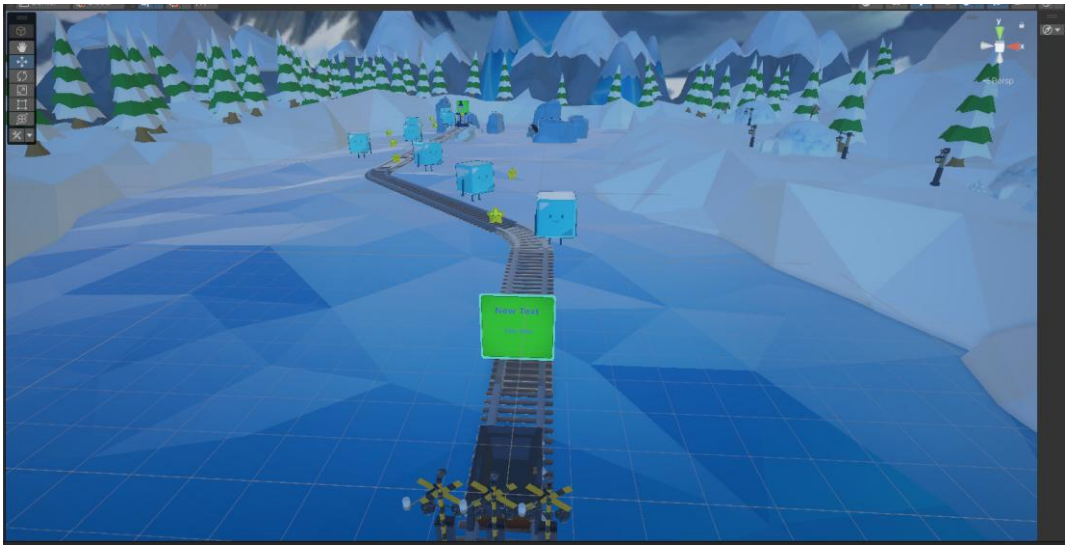

Figure 3.4: Level 1: Minecart Ride

3.3.3 Level 2 – Snowball Dodge

The second level builds on reflexes and reaction time. The player remains stationary while snowballs are launched by a cannon. Using body-tracking data, the system detects quick side dodges to avoid the projectiles.

Scripts such as SnowballSpawner and DodgeValidationSystem manage projectile instantiation, trajectory, and collision detection. Each successful dodge rewards the player with score points and playful sound effects, reinforcing engagement through positive feedback.

Figure 3.5: Level 2: Snowball Dodge

3.3.4 Level 3 – Rope Bridge Crossing

The final level focuses on balance control, representing the project's therapeutic and educational intent. The player must walk across a suspended rope bridge while maintaining equilibrium using small torso adjustments.

The PlayerBalanceController processes tilt and positional data to calculate a stability index, while the RopeBridgeChallenge script monitors progress along the bridge. If the player leans excessively to one side, the system simulates a fall using camera tilt and vibration feedback; maintaining balance until the timer ends leads to the penguin's rescue.

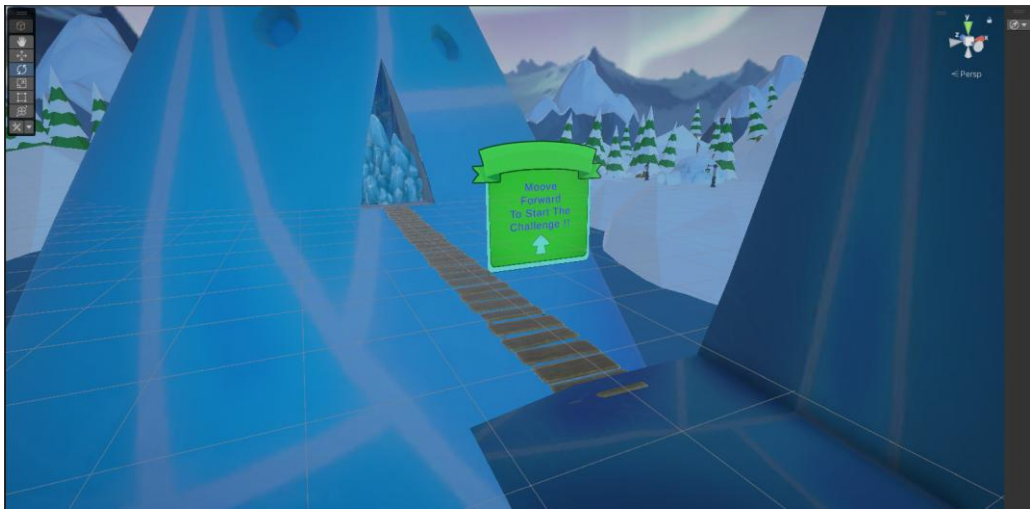

Figure 3.6: Level 3: Rope Bridge Crossing

3.4 Core Systems Integration

The core systems of *Penguin Parade* ensure the coherence between all gameplay elements from level logic to body tracking and user feedback. These systems form the technical backbone of the project, enabling modularity, scalability, and fluid communication between scripts.

3.4.1 Level GameManagers

Each level in Penguin Parade contains its own LevelGameManager, responsible for managing scene-specific mechanics and level transitions. The LevelGameManager is a dedicated controller for each stage of the game and governs all related logic such as player movement, obstacle spawning, and goal completion. Upon completion of a level, the LevelGameManager triggers the transition to the next stage, ensuring smooth progression through the game.

For instance, in Level 1 (Minecart Ride), the manager controls the minecart's path and obstacle interactions. In Level 2 (Snowball Dodge), it monitors the snowball cannon spawn system and dodging mechanics. In Level 3 (Rope Bridge Crossing), it handles the balance logic and final success criteria. Each manager is responsible for its specific stage, ensuring clean, isolated gameplay mechanics.

Once a level's objective is met, the LevelGameManager of the current level passes control to the LevelGameManager of the next stage, maintaining the game's flow and cohesion.

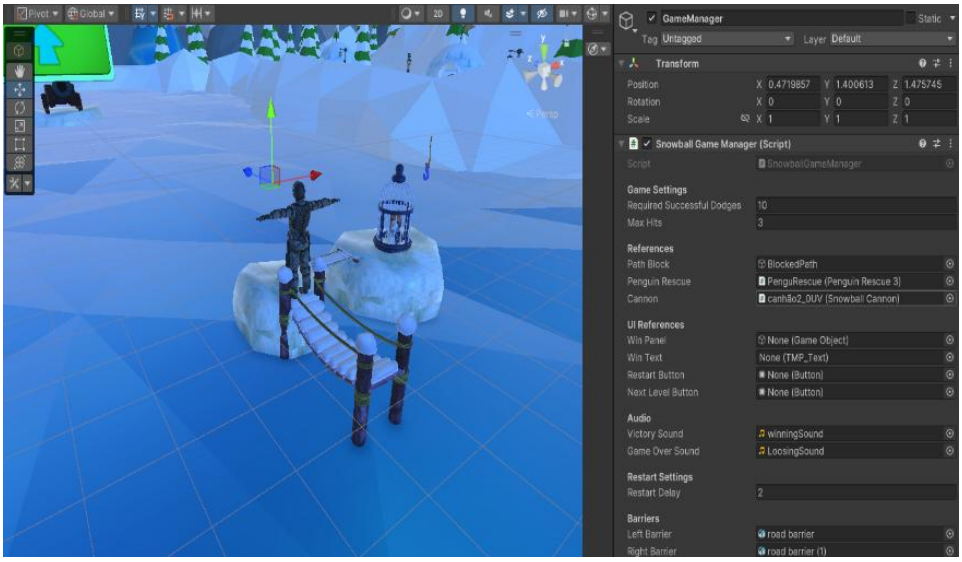

Figure 3.7: Example of LevelGameManager scene hierarchy in Unity

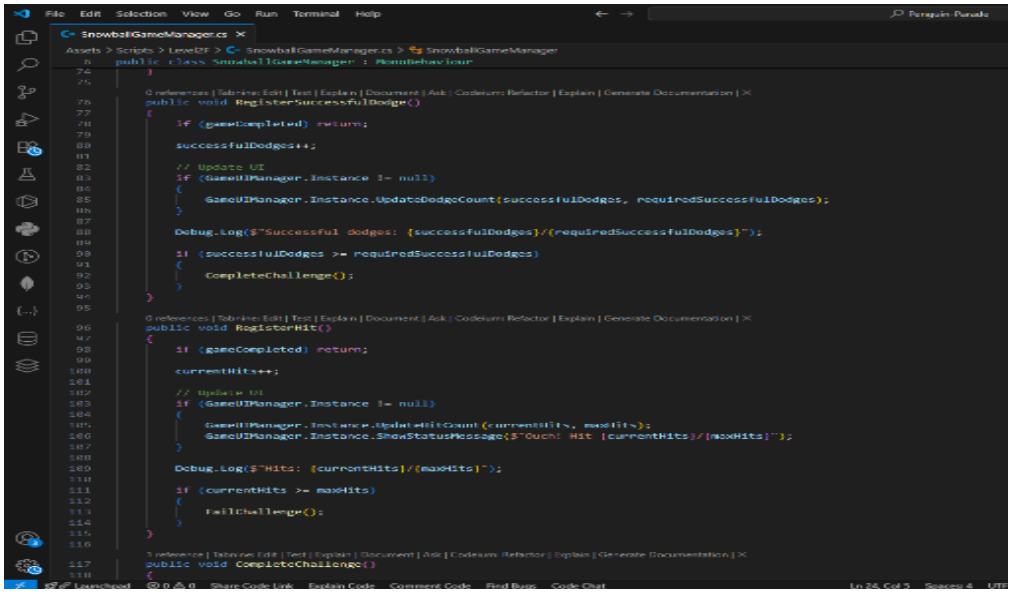

Figure 3.8: Example of LevelGameManager script

3.4.2 Body Tracking Integration (Meta Movement SDK)

The Meta Movement SDK is used to capture real-time body posture data, specifically from the player's hips and torso. A calibration phase sets a neutral position, allowing for accurate tracking of leaning, balancing, and dodging. Raw tracking data is processed using smoothing algorithms to stabilize movements and avoid unintentional actions. This ensures that the player's real-world movements are accurately reflected in the game, driving interactions across all levels.

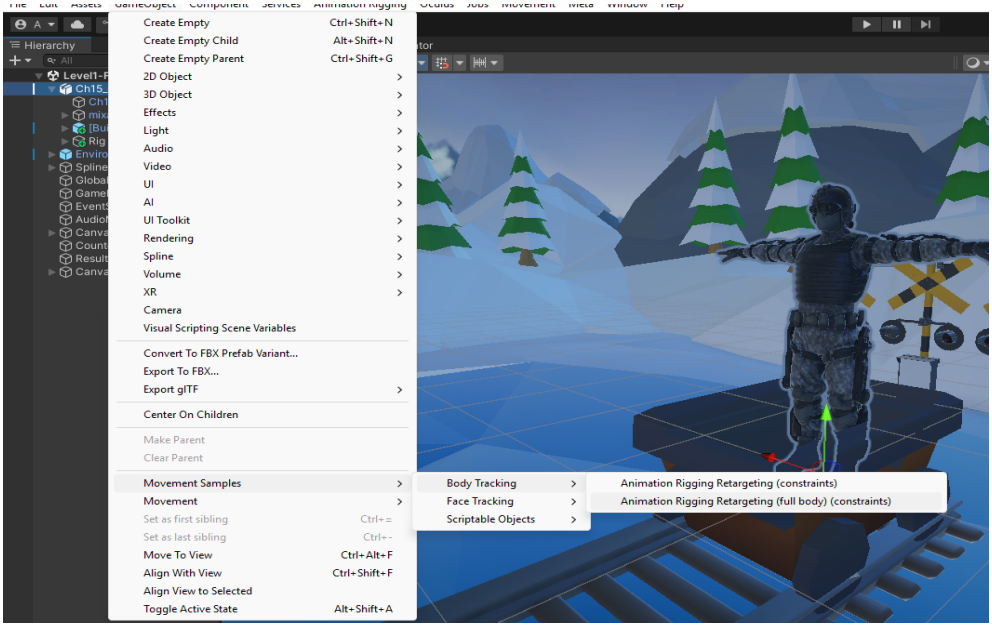

Figure 3.9: Integration of Meta Movement SDK for posture and lean detection

3.4.3 UI and Feedback Systems

The interface for Penguin Parade was designed in world-space to maintain immersion. Essential elements like score display, instructions, and timers were positioned ergonomically to avoid visual strain.

Audio feedback for each action is provided through short sound effects and voice cues, generated using Audacity and ElevenLabs. These cues inform the player of their progress and reinforce actions such as successful dodges, failed attempts, and level completion.

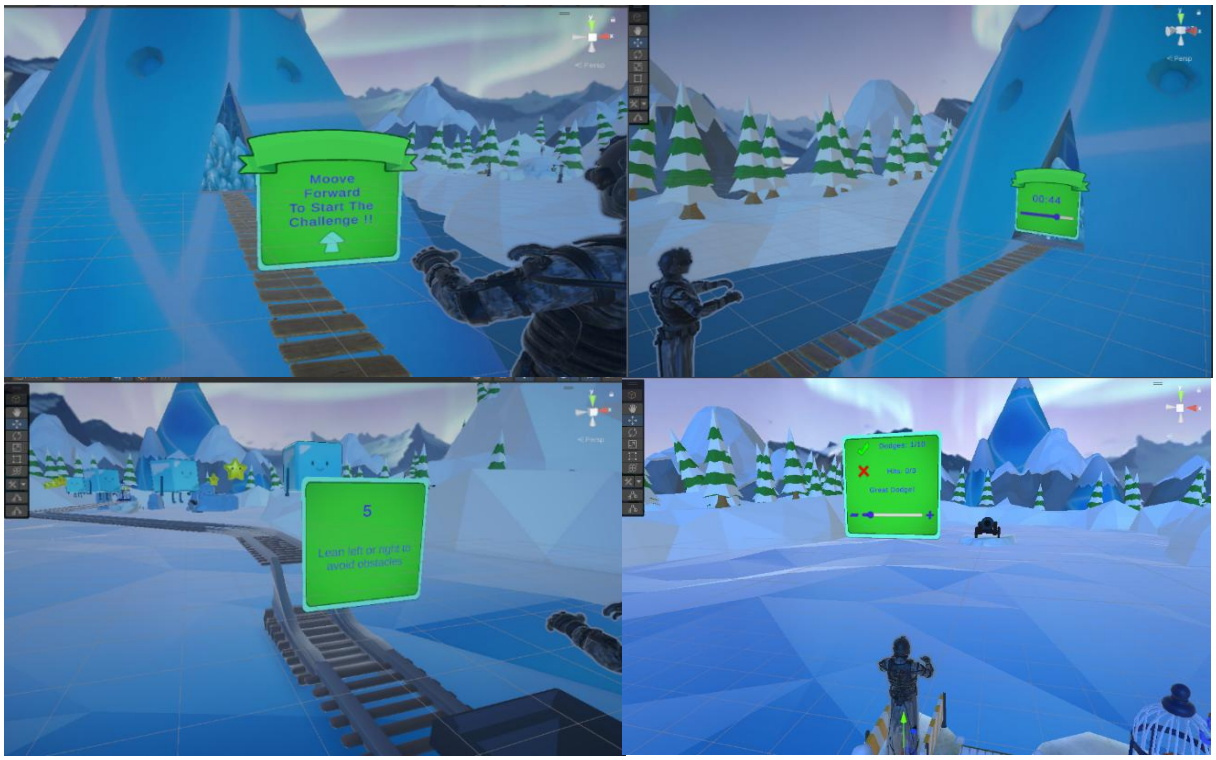

Figure 3.10: Example of world-space UI and player feedback in the game

3.4.4 Event-Driven Communication

All systems in Penguin Parade gameplay, UI, audio, and scoring are interconnected through event-driven communication. This structure ensures smooth transitions, optimized performance, and easy maintenance.

For example, when a successful dodge is detected, the following actions are triggered:

1. The ScoreManager increases the player's score.
2. The AudioManager plays a feedback sound.
3. The UIManager updates the score on the screen.

By using events, this architecture avoids redundant polling and optimizes the game's performance.

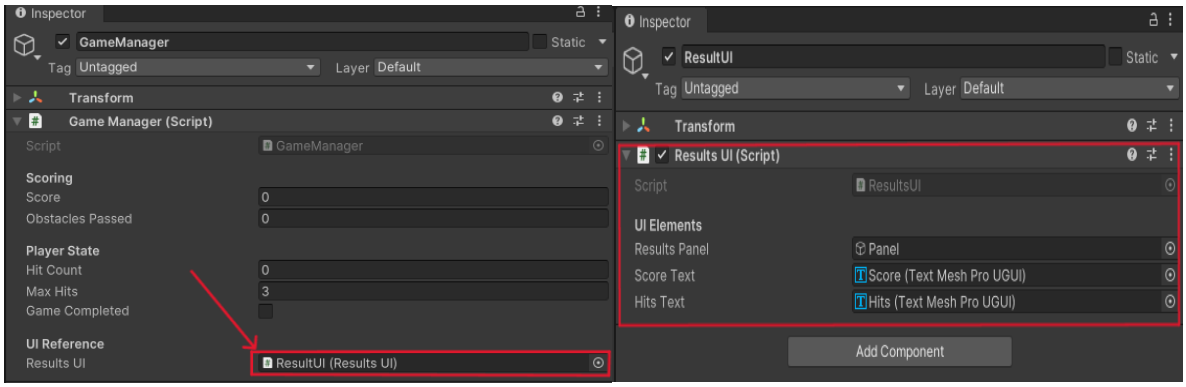

Figure 3.11: Example of the Event flow between GameManager and ResultUI

3.5 Inventory of Assets

The development of Penguin Parade relied entirely on externally sourced assets to create an immersive and functional VR experience. These assets were primarily downloaded from Sketchfab, a platform that provides free and licensed 3D models, which were then optimized and adapted for Unity. Blender was used only for minor adjustments such as cleaning up meshes, aligning pivots, and ensuring proper scaling.

3.5.1 Imported Assets

The majority of the game's 3D models, textures, and props were imported directly from online repositories, such as Sketchfab. The imported assets were carefully selected based on their visual consistency and license permissions, ensuring both technical compatibility with VR and consistency in style across all levels.

The key assets included:

- Environmental models such as snow-covered ground, trees, rocks, and obstacles for the levels.
- Character models for NPCs and the penguin, used to populate the scenes.
- Game props like interactive signs (for level selection), snowballs, and minecart obstacles.

3.5.2 Blender Modifications

While most of the assets were sourced externally, Blender was used to make necessary tweaks for performance and compatibility. These modifications included:

- ❖ Optimizing polygon counts for smoother performance on the Meta Quest 3.
- ❖ Rescaling and adjusting pivots to ensure correct placement and interaction in Unity.
- ❖ Cleaning up meshes to remove unnecessary details that could impact VR performance.
- ❖ UV mapping adjustments for a more efficient application of textures.

These changes ensured the imported assets were both VR-ready and performance-optimized for smooth gameplay on the Meta Quest 3.

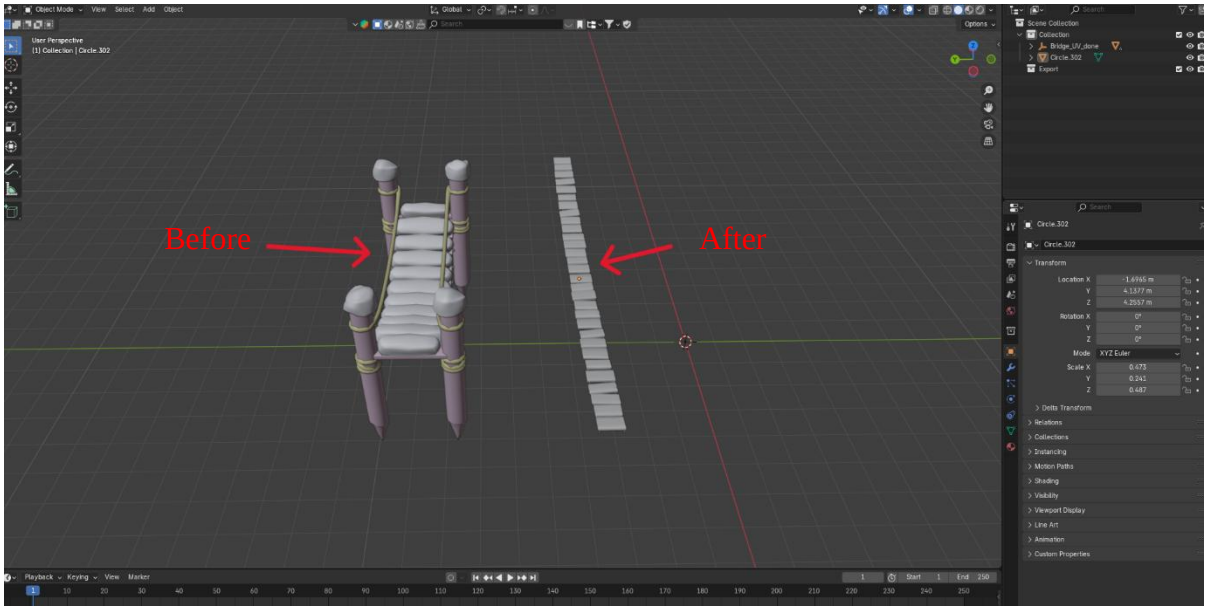

Figure 3.12: Example of Sketchfab asset before and after Blender modifications

3.5.3 Audio Assets

In addition to 3D models, audio assets were essential to create a fully immersive environment. These assets were sourced and processed as follows:

- ❖ Audacity: Used for recording, editing, and refining sound effects (success cues, ambient sounds, and impact noises).
- ❖ ElevenLabs: An AI-based tool used to generate text-to-speech audio files for NPC dialogue (e.g., "Welcome to the Snowball Dodge level").

All audio was spatialized in Unity to ensure that sounds came from the appropriate in-game locations, enhancing the sense of immersion.

Figure 3.13: Example of audio assets in Unity

3.5.4 Asset Optimization

Optimization was key to ensuring smooth gameplay performance on the Meta Quest 3. Imported assets underwent several optimization steps:

- ❖ Low-poly modeling for models to reduce the load on the GPU.
- ❖ Texture compression and the use of texture atlases to save memory.
- ❖ Baked lighting to eliminate the need for real-time lighting calculations, thus reducing CPU/GPU load.

This careful optimization ensured that all assets worked together to provide a smooth and immersive VR experience while meeting the performance requirements of the Meta Quest 3.

3.6 Interaction & Mechanics Details

The core of *Penguin Parade* lies in its interactive gameplay mechanics**,** where the player's real-world body movements directly influence in-game actions. The use of the Meta Movement SDK enables precise body tracking, allowing the player to physically engage with the environment using intuitive gestures. The game mechanics were designed to be simple yet engaging**,** ensuring both ease of use and effective pedagogical outcomes.

3.6.1 Leaning and Dodging (Level 1 – Minecart Ride)

The first mechanic introduced is leaning. In the Minecart Ride level, the player must lean left or right to avoid obstacles that appear on the track. The PlayerLeanDetector script, powered by the Meta Movement SDK, captures the player's lateral lean and compares it against

predefined thresholds. If the player leans in the correct direction within the allowed time frame, they successfully dodge the obstacle, and their score is updated.

This mechanic trains the player's reflexes and spatial awareness, requiring them to act quickly but without causing discomfort.

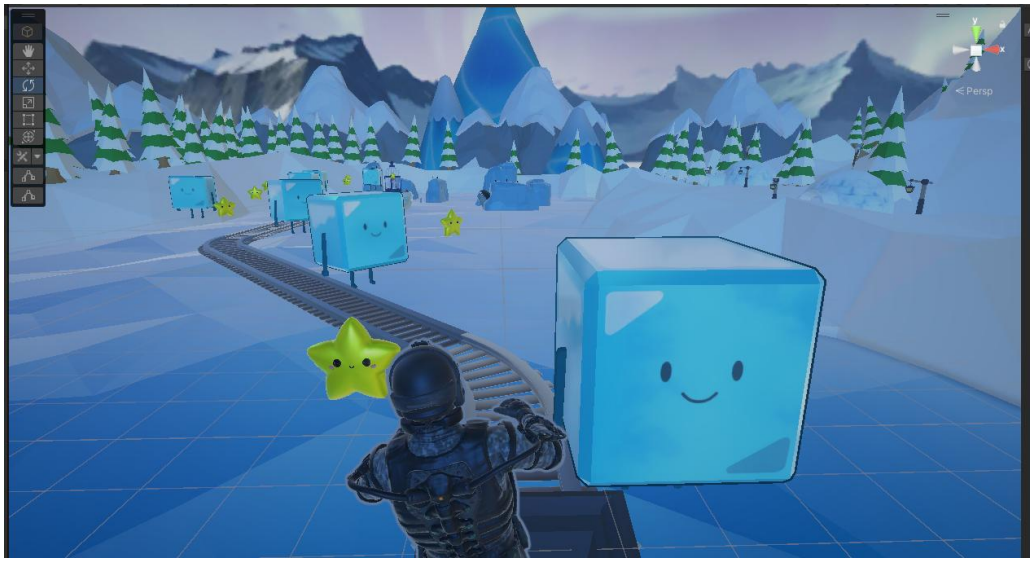

Figure 3.14: Example showing leaning interaction and obstacles.

3.6.2 Crouching and Dodging (Level 2 – Snowball Dodge)

The Snowball Dodge level builds on this with a focus on reaction time and full-body engagement. In this level, snowballs are launched at the player from various cannons. The player must use leaning or crouching to avoid them. The DodgeValidationSystem detects whether the player has successfully avoided the snowball. This level encourages the player to respond quickly to visual and auditory cues, enhancing their reflexes while maintaining comfort.

Figure 3.15: Example showing dodging snowballs via leaning.

3.6.3 Balance Control (Level 3 – Rope Bridge Crossing)

The final level introduces balance control, one of the most critical mechanics for this project. In the Rope Bridge Crossing, the player must walk across a narrow bridge, maintaining their balance using small, controlled body movements. The PlayerBalanceController2 monitors hip tilt and torso rotation to track stability. If the player leans too far in any direction, the system simulates a fall by shaking the camera and playing a failure sound.

This mechanic simulates real-world balance challenges, training the player's coordination and postural control.

Figure 3.16: Example showing balance control and the player's posture.

3.6.4 Feedback Systems and Rewards

Each mechanic is paired with real-time feedback to reinforce correct actions. Visual and auditory cues provide immediate responses to player interactions. For example:

- Audio feedback (success or failure) is generated using the ElevenLabs TTS system or custom sound effects in Unity.
- Visual feedback in the form of color flashes, score updates, and screen shakes is triggered when the player successfully dodges or falls.

The combination of audio cues, and dynamic visual feedback ensures the player receives constant input on their actions, keeping them engaged and informed throughout the experience.

3.7 Validation and Testing

Throughout the development of Penguin Parade, a rigorous validation process was employed to ensure that each feature and interaction functioned correctly and met the performance standards for VR gameplay on the Meta Quest 3. Testing was conducted iteratively, with each new feature tested immediately upon implementation to detect any issues early on.

3.7.1 Functional Testing

Each game mechanic was tested in isolation to ensure that it met its design goals. For example, the dodging mechanics in the Minecart and Snowball levels were tested for responsiveness, ensuring the player’s movements (leaning, crouching, etc.) were correctly interpreted by the Meta Movement SDK.

Similarly, the balance mechanic in the Rope Bridge level was thoroughly tested to confirm the player's posture and movements were accurately tracked. Feedback systems, including audio cues and score updates, were also tested to ensure timely, clear responses to player actions.

3.7.2 Performance Testing

To ensure smooth gameplay on the Meta Quest 3, performance profiling was conducted regularly using Unity’s Profiler.

Key metrics such as frame rate (target 72 FPS), GPU usage, and memory consumption were monitored and optimized. Optimization techniques, such as occlusion culling and baked lighting, were implemented to keep the game running at a stable frame rate without sacrificing visual quality.

Testing was done directly on the Quest 3 to replicate real-world performance conditions, ensuring that the game was comfortable for users and free of lag or stutter.

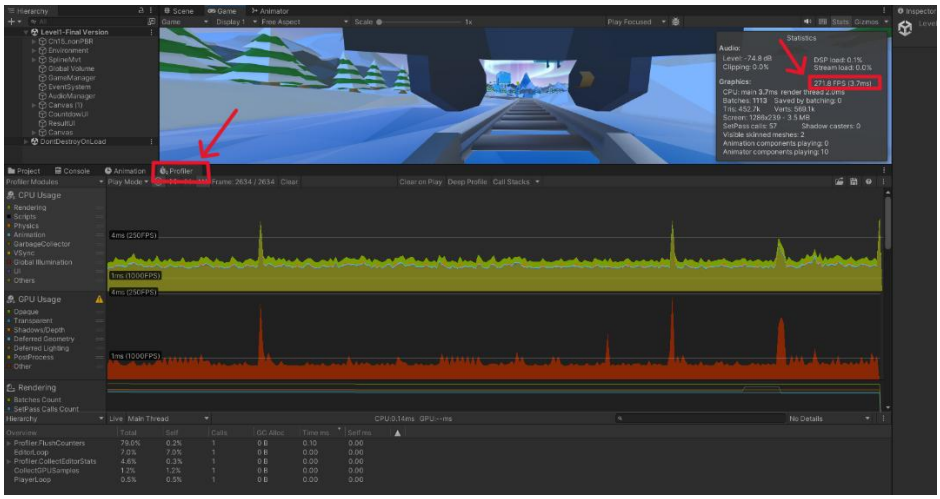

Figure 3.17: Unity Profiler showing frame rate and performance metrics

3.7.3 User Feedback

In addition to technical testing, user feedback played a crucial role in validating the usability and comfort of the experience.

Informal playtests were conducted with peers and colleagues, where they were asked to interact with the game, perform various actions, and provide feedback on the intuitiveness of the controls and overall comfort during play. Issues like motion sickness or difficulty in balancing were addressed by adjusting the sensitivity of body tracking and modifying certain visual effects to enhance the player's comfort.

This feedback also influenced the refinement of the main menu interaction system, ensuring it was easy for users to navigate and select levels.

3.7.4 Debugging and Iterative Testing

Each testing phase was followed by debugging sessions, where issues identified during functional and performance testing were tracked and resolved in GitLab.

This process was iterative, meaning that as new features were implemented, the game was tested again to ensure no new issues were introduced.

The testing cycle continued throughout development, leading to a stable version of the game ready for final delivery.

3.8 Final Build and Delivery

Once the core mechanics were fully implemented and functional, the final phase of development focused on optimizing the game for performance and ensuring the experience was both fluid and immersive for users. This phase involved several key adjustments to enhance both visual quality and user comfort on the Meta Quest 3, where performance constraints are more stringent than on PC VR platforms.

3.8.1 Performance Optimization

As the game neared completion, various optimization techniques were applied to ensure smooth gameplay at the target 72 FPS frame rate.

- Texture Compression was used to reduce the load on memory while maintaining visual fidelity.
- Occlusion Culling was applied to prevent the rendering of hidden objects, thereby reducing GPU usage.
- Instead of using real-time lighting, a skybox/panoramic material was applied to simulate natural lighting, providing a consistent and immersive environment without the need for dynamic light sources.
- Low-Poly Assets were prioritized, ensuring that models and textures were optimized for VR performance without sacrificing essential visual quality.

These techniques were tested continuously on the Meta Quest 3 to ensure real-world performance without compromising the game's visual or interactive quality.

3.8.2 User Comfort and Experience Refinements

In addition to performance optimizations, user comfort was a key consideration, particularly due to the full-body engagement required by the gameplay. Adjustments were

made to the body movement mechanics to ensure smooth and natural interactions, such as dodging, leaning, and balancing.

To prevent discomfort, the balance detection system was fine-tuned to provide subtle visual cues (like camera tilt) rather than excessive movement. The field of view (FOV) and camera settings were optimized to reduce visual fatigue and enhance immersion, ensuring the player could comfortably engage in the game for extended periods.

3.8.3 Final Build Export and Delivery

Once the game was fully optimized and the gameplay mechanics refined, the final step was preparing the build for Meta Quest 3. Unity's Oculus XR plugin (instead of OpenXR, since the Meta Movement SDK is optimized for Oculus) was used to ensure compatibility with the Quest's hardware. Player settings were configured for Android, including resolution, refresh rate, and memory allocation.

The game was thoroughly tested on the Quest 3 to ensure stability and smooth gameplay. After final verification, the APK build was exported and installed onto the device for real-world playtesting.

3.9 Conclusion

This chapter provided a comprehensive overview of the realization and implementation process behind Penguin Parade. From asset preparation and optimization to core systems integration, each step was carefully designed to ensure a smooth and immersive experience for the player on the Meta Quest 3.

The key focus was on optimization for performance, comfort, and VR immersion, using techniques such as texture compression, occlusion culling, and skybox lighting to ensure smooth gameplay at 72 FPS. Equally important was the fine-tuning of user interactions, including body tracking, balance control, and feedback systems, all of which contributed to a natural and engaging experience.

By the end of this phase, the game was fully optimized and tested, and the final build was exported as an APK for real-world playtesting on the Meta Quest 3. This iterative process ensured that the game met the desired technical, pedagogical, and immersive standards, setting a solid foundation for future iterations or potential extensions.

4. General Conclusion and Perspectives

This report has outlined the work completed as part of my internship project, which involved the design and development of Penguin Parade, a virtual reality (VR) game aimed at improving balance, reflexes, and coordination for children, particularly those with motor coordination challenges.

The project was driven by the goal of creating an engaging and educational VR experience that would support children in improving their motor skills in a fun, interactive, and immersive way. By integrating body tracking via the Meta Movement SDK, Penguin Parade allows players to physically interact with the game, performing actions like dodging obstacles, leaning, and balancing, which helps strengthen reflexes and body awareness.

The development process, carried out autonomously within Inherited Games Studio, allowed me to explore various aspects of a complete VR project, including game mechanics design, 3D asset integration, body-tracking calibration, and user experience optimization. From the initial concept to the final build, the project involved overcoming challenges such as optimizing performance for standalone hardware and refining the user interface (UI) for younger audiences.

Through this internship, I gained valuable experience in agile development practices, performance optimization, and project management, while also expanding my skills in both technical and creative areas. The project was completed within the two-month timeframe, resulting in a functional and optimized prototype for therapeutic use.

Looking ahead, several future directions for Penguin Parade are possible. New levels and challenges could be added to target additional motor skills, such as jumping or climbing. The game could also benefit from real-time progress tracking, providing data on player performance to help therapists monitor progress over time. Additionally, a multiplayer mode could encourage social interaction, providing a shared experience that fosters cooperation and peer learning.

Expanding the game to other platforms like mobile or desktop would broaden its accessibility, allowing it to reach a larger audience, including schools and rehabilitation centers that may not have access to VR hardware. Furthermore, Penguin Parade could serve as a foundation for other educational VR experiences, potentially addressing areas like physical education, therapy, and even team-building activities for children.

In conclusion, Penguin Parade represents a successful implementation of VR as a therapeutic and educational tool, offering a solid foundation for further development and adaptation in the field of child rehabilitation.

References

[1] LinkedIn page Inherited Games Studio: https://www.linkedin.com/company/inherited-games-studio

[2] Beat Saber: https://beatsaber.com

[3] Richie's Plank Experience: https://www.meta.com/experiences/pcvr/richies-plank-experience/1388204261277129/

[4] The Climb 2: https://www.meta.com/experiences/the-climb-2/2617233878395214/

[5] Scrum Framework Overview: https://www.pm-partners.com.au/insights/the-agile-journey-a-scrum-overview/

[6] ClickUp: https://app.clickup.com

[7] Meta Quest 3 : https://www.meta.com/quest/quest-3/

[8] Unity: https://unity.com/games

[9] Meta XR All-in-One SDK: https://assetstore.unity.com/packages/tools/integration/meta-xr-all-in-one-sdk-269657

[10] Meta Movement SDK: https://developers.meta.com/horizon/documentation/unity/move-overview/

[11] Blender: https://www.blender.org/about/logo/

[12] Sketchfab: https://sketchfab.com/feed

[13] GitLab: https://about.gitlab.com/

[14] Lucidchart: https://www.lucidchart.com/pages

[15] Audacity: https://www.audacityteam.org/

[16] ElevenLabs: https://elevenlabs.io/